Project 3: Water Treatment Network Design

Jorge Urruela, Process Systems Modeling

DISCLAIMER: I mentioned in my proposal that I would solve the concentration based problem. However, I used the data in the flow based code and created my own contaminant/water concentration based problem, instead of directly replicating the concentration based problem.

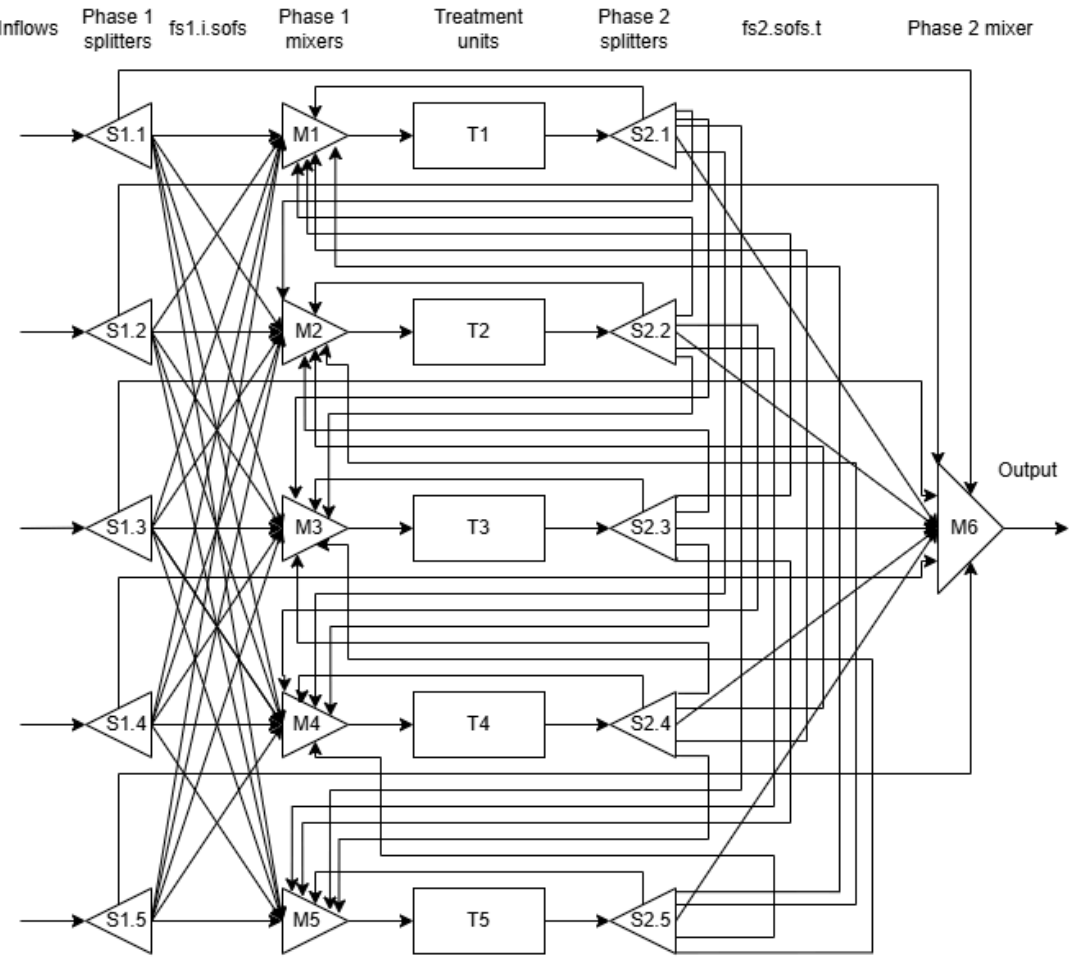
Problem description (Ruiz, 2009)

There are 5 water streams with 4 contaminants that need to be purified with at most 5 different treatment units. So, first, stage 1 splitters are placed so every incoming stream can feed every mixer, for one which goes to the output and the rest feed a treatment unit each. Treatment units feed the second set of splitters, which once again feed the mixers. Water quaility is mantained by keeping flows of contaminants under a certain range, each of which in this case is set under 3. In other words, if the system works as it should, the molar flow of each contaminant should be 3 or less.

Units are not specified, which is not a problem as long as they are consistent. For instance, it could be given in mol, kmol, lbmol/s or really any molar or molar flow units, as long as they are consistent throughout the entire process.

Figure 1 represents the superstructure of the problem

Figure 1: Problem Superstructure



The problem consists of 3 sets and 2 subsets represented as subindicies, 7 variables represented by lowercase letters, 7 parameters represented by either uppercase letters or greek letters, and 11 equations. I will start by solving the code (which is described), and then explain the equations as some parameters must be printed from the code.

In [26]:
```
import gamspy as gp
m = gp.Container()
#sets
inflows = gp.Set(m,"inflows",records = ["1","2","3","4","5"]) #there is one splitte
t = gp.Set(m,"treatmentunits",records = ["1","2","3","4","5"]) #there is one mixer
comps=gp.Set(m,"components",records = ["A","B","C","D","water"])
conts = gp.Set(m,"contaminants",domain=comps,records = ["A","B","C","D"])
mi = gp.Set(m,"mixers",records = ["1","2","3","4","5","6"])#all mixers
m1 = gp.Set(m,"mixers1",domain=mi,records = ["1","2","3","4","5"])# first set of mi
m2 = gp.Set(m,"mixers2",domain=mi,records = ["6"])# second set of mixers
mtlink = gp.Set(m,"mtlink",domain=[m1,t],records=[("1","1"),("2","2"),("3","3"),("4
#variables
fs1 = gp.Variable(m,"fs1",domain=[comps,inflows,mi],type="positive") #flows coming
fm = gp.Variable(m,"fm",domain=[comps,mi],type="positive") #flows coming out of the
ft = gp.Variable(m,"ft",domain=[comps,t],type="positive") #flows coming out of the
fs2 = gp.Variable(m,"fs2",domain=[comps,t,mi],type="positive") #flows coming out of
ctu = gp.Variable(m,"ctu",domain=[t],type="positive") #cost of treatment units
ytu = gp.Variable(m,"ytu",domain=[t],type="binary") #whether to use or not the trea
cost = gp.Variable(m,"cost") #total cost, objective variable
```

```
#parameters
# flows going into the process
inlets = gp.Parameter(m,"inlets",domain=[comps,inflows],records=[("A","1",1),("B","
                                                                 ("A","2",2),("B","
                                                                 ("A","3",1),("B","
                                                                 ("A","4",3),("B","
                                                                 ("A","5",2),("B","
# fraction removed of each contaminant in each treatment unit
effs = gp.Parameter(m,"effs",domain=[comps,t],records=[("A","1",0.95),("B","1",0.95
                                                       ("A","2",0.80),("B","2",0.80
                                                       ("A","3",0.00),("B","3",0.60
                                                       ("A","4",0.00),("B","4",0.60
                                                       ("A","5",0.00),("B","5",0.60
# contaminant limits
contlims = gp.Parameter(m,"contlims",domain=[conts],records=[("A",3),("B",3),("C",3
# Minimum water flow limits for equipments
eqlims = gp.Parameter(m,"eqlims",domain=[t],records=[("1",5),("2",3),("3",4),("4",3
# big M (based on the sum of all incoming flows of water)
bigM=60
# cost parameters, based on the molar flow of water passing through the equipment
theta = gp.Parameter(m,"theta",domain=[t],records=[("1",1500),("2",1000),("3",4000)
beta = gp.Parameter(m,"beta",domain=[t],records=[("1",8000),("2",8000),("3",8000),(
gamma = gp.Parameter(m,"gamma",domain=[t],records=[("1",0),("2",0),("3",0),("4",0),
#equations
    #constraints
        #mass balances
mbs1 = gp.Equation(m,"mbs1",domain = [comps,inflows]) #mass balances in the first s
mbs1[comps,inflows] = inlets[comps,inflows] == gp.Sum(mi,fs1[comps,inflows,mi])
mbs2 = gp.Equation(m,"mbs2",domain = [comps,t]) #mass balances of the second set of
mbs2[comps,t] = ft[comps,t] == gp.Sum(mi,fs2[comps,t,mi])
mbm = gp.Equation(m,domain = [comps,mi]) #mass balances for the mixers
mbm[comps,mi]= gp.Sum(t,fs2[comps,t,mi])+gp.Sum(inflows,fs1[comps,inflows,mi]) == f
mbt = gp.Equation(m,"mbt",domain = [comps,t]) #mass balance for the treatment units
mbt[comps,t]= gp.Sum(m1,fm[comps,m1].where[mtlink[m1,t]])*(1-effs[comps,t]) == ft[c
        #restrictions
qual = gp.Equation(m,domain = [conts]) #quality constraint, contaminants must be un
qual[conts] = gp.Sum(m2,fm[conts,m2])<=contlims[conts]
minsize = gp.Equation(m,"minsize",domain=[t])#equipment chosen should have at least
minsize[t]=eqlims[t]*ytu[t]<=gp.Sum(m1, fm["water", m1].where[mtlink[m1, t]])
bigMeq = gp.Equation(m,"bigMeq",domain=[t]) #bigM constraint, defining ytu
bigMeq[t] =gp.Sum(m1, fm["water", m1].where[mtlink[m1, t]]) <=bigM * ytu[t]
indcost = gp.Equation(m,"indcost",domain=[t]) #defining the cost of individual trea
indcost[t] = ctu[t] == theta[t] *gp.Sum(m1, fm["water", m1].where[mtlink[m1, t]]) *
eqconcs1 = gp.Equation(m,"eqconcs1",domain=[conts,inflows,mi]) #ensure equal concen
eqconcs1[conts, inflows, mi] = fs1[conts, inflows, mi] * inlets["water", inflows]==
eqconcs2 = gp.Equation(m,"eqconcs2",domain=[conts,t,mi]) #ensure equal concentratio
eqconcs2[conts, t, mi] = fs2[conts, t, mi] * ft["water", t]==ft[conts, t] * fs2["wa
    #objective equation
obj = gp.Equation(m,"obj")
obj[...] = cost == gp.Sum(t,ctu[t])
model = gp.Model(m,"MINLP",equations=[obj,mbs1,mbs2,mbm,mbt,qual,minsize,bigMeq,ind
model.solve()
```

```
Cost: $348337.0416795783
```

In [33]:
```
# This is just to display the parameters and results nicely, by ChatGPT
def nice_param_table(param):
    df = param.records
    nicetable = df.pivot(
        index=df.columns[0],
        columns=df.columns[1],
        values="value")
    display(nicetable)
```

Problem statement

Sets (represented by subindicies)

Note: I discovered later that t can be described as a subset of m, which makes a slight difference in the code when compared to the previuos block but the problem becomes much easier to explain.

m: mixers , $\mathrm{m} \in \{0, 1, 2, 3, 4, 5, 6\}$

i: input flows, $\mathrm{i} \in \{0, 1, 2, 3, 4, 5\}$

c: components, $\mathrm{c} \in \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{water}\}$

Subsets

t: treatment units, $\mathrm{t} \in \{0, 1, 2, 3, 4, 5\}$ and is a subset of m

conts: contaminants, $\mathrm{t} \in \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}\}$ and is a subset of c

Variables

Positive Variables

$\mathrm{fs1}_{\mathrm{c,i,m}}$: Flows coming out of each splitter in the first set

$\mathrm{fs2}_{\mathrm{c,t,m}}$: Flows coming out of each splitter in the second set

$\mathrm{fm}_{\mathrm{c,m}}$: Flows coming out of each mixer

$\mathrm{ft}_{\mathrm{c,t}}$: Flows coming out of each treatment unit

ct_{t}: Cost of each treatment unit

Binary variables

yt_{t}: Whether or not to use a treatment unit

Free variables

cost: Total cost, which is also the variable of interest

Parameters

M: Big-M, $M = 60$

$I_{c,i}$: Each flow with its respective composition entering the process

Table 1: $I_{c,i}$

In [34]:
```
nice_param_table(inlets)
```

inflows	1	2	3	4	5
components					
A	1.0	2.0	1.0	3.0	2.0
B	1.0	0.0	0.0	1.0	2.0
C	1.0	0.0	1.0	1.0	0.0
D	1.0	1.0	2.0	0.0	0.0
water	10.0	15.0	5.0	20.0	10.0

$E_{c,t}$: Removal efficiency of each treatment unit per compound

Table 2: $E_{c,t}$

In [35]:
```
nice_param_table(effs)
```

treatmentunits	1	2	3	4	5
components					
A	0.95	0.8	0.00	0.00	0.00
B	0.95	0.8	0.60	0.60	0.60
C	0.60	0.6	0.95	0.80	0.85
D	0.00	0.0	0.95	0.85	0.80
water	0.00	0.0	0.00	0.00	0.00

Q_{conts}: Quality constraint for the output flow

Table 3: Q_{conts}

In [36]:
```
contlims.records
```

Out[36]:

	contaminants	value
0	A	3.0
1	B	3.0
2	C	3.0
3	D	3.0

Table 4: Parameters of the treatment units

Parameter	Description	T1	T2	T3	T4	T5
L_t	Lower limit of operation of the treatment unit	5	3	4	3	1
θ_t	Economies of scale cost component coefficient	1500	1000	4000	3000	3000
β_t	Linear cost component coefficient	8000	8000	8000	8000	8000
γ_t	Fixed cost componenet coefficient	0	0	0	0	0

Functions

Objective Equation

$\text{cost} = \sum_t \text{ct}_t$

Subject to:

Mass balances

First set of splitters:

$\text{I}_{c,i} = \sum_m \text{fs1}_{c,i,m}$

Second set of splitters:

$\text{ft}_{c,t} = \sum_m \text{fs2}_{c,t,m}$

Mixers:

$\text{fm}_{c,m} = \sum_i \text{fs1}_{c,i,m} + \sum_t \text{fs2}_{c,t,m}$

Treatment units:

$\text{ft}_{c,t} = \text{fm}_{c,t} * (1 - \text{E}_{c,t})$ gp.Sum(m1,fm[comps,m1].where[mtlink[m1,t]])*(1-effs[comps,t]) == ft[comps,t]

Restrictions

Quality constraint:

$$fm_{conts,6} \leq Qconts$$

Minimal size:

$$L_t * yt_t <= fm_{water,t}$$

Big-M constraint:

$$fm_{water,t} \leq M * yt_t$$

Individial cost of treatment units (non-linear):

$$ct_t = \theta_t * (\sum_t fm_{water,t})^{0.7} + \beta_t * \sum_t fm_{water,t} + \gamma_t * yt_t$$

Concentration is mantained for outputs of first splitters (after multiplying each side of the equality by the water flow):

$$fs1_{conts,i,m} * I_{water,i} = I_{conts,i} * fs1_{water,i,m}$$

Same thing for the second set of splitter, but this one is nonlinear:

$$fs2_{conts,t,m} * ft_{water,i} = ft_{conts,t} * fs2_{water,t,m}$$

Results

In [40]:
```
print("Model status:", model.status)
print("Solve status:", model.solve_status)
print(f"Operation cost: ${cost.toValue():.2f}")
```

```
Model status: ModelStatus.Integer
Solve status: SolveStatus.NormalCompletion
Operation cost: $348337.04
```

Table 5: Chosen treatment units

In [45]:
```
ytu.records
```

Out[45]:

	treatmentunits	level	marginal	lower	upper	scale
0	1	1.0	-0.000000	0.0	1.0	1.0
1	2	0.0	-0.000000	0.0	1.0	1.0
2	3	0.0	-0.000000	0.0	1.0	1.0
3	4	1.0	28531.105488	0.0	1.0	1.0
4	5	0.0	-0.000000	0.0	1.0	1.0

Table 6: Detailed treatment units cost

In [52]:
```
ctu.where["value"!=0].records
```

Out[52]:

	treatmentunits	value
0	1	317864.03384
1	4	30473.00784

Table 7: Nonzero flow destination of each splitter of the first set

In [61]:
```
nice_param_table(gp.Sum(comps,fs1[comps,inflows,mi]).where["value"!=0])
```

mixers	1	4	6
inflows			
1	NaN	NaN	14.000000
2	2.842105	NaN	15.157895
3	3.705882	5.294118	NaN
4	25.000000	NaN	NaN
5	14.000000	NaN	NaN

Table 8: Nonzero flow destination of each splitter of the second set

In [62]:
```
nice_param_table(gp.Sum(comps,fs2[comps,t,mi]).where["value"!=0])
```

mixers	1	2	4	6
treatmentunits				
1	NaN	NaN	NaN	39.600179
2	NaN	6.018531e-36	NaN	NaN
4	3.818727	NaN	0.076375	NaN

Table 9: Flows entering each treatment unit

In [57]:
```
nice_param_table(fm[comps,m1].where["value"!=0])
```

mixers1	1	2	4
components			
A	6.315789	NaN	0.600000
B	3.000000	NaN	NaN
C	1.527559	4.557050e-17	0.590551
D	1.154945	NaN	1.179941
water	37.368421	NaN	3.000000

Discussion

A different approach to the ones presented in the reference (Ruiz, 2009), was given, where a combination of the two was taken. Here, a solvent concentration approach towards the flow-based data was taken.

This is a MINLP because of a few reasons. First, the cost is in a nonlinear basis. Then, all approaches have at least one nonlinear equation, to either solve for the concentration, the split of flows or mantain concentrations in the splits of the treatment units. There may be ways to solve this, but would greatly complicated and, if to start, the cost is nonlinear, the problem is nonlinear too.

But even with these complications, either one of the 3 approaches gave equivalent solutions, even when working in another platform like GAMS vs GAMSPy and using different algorithms.

The model is suggesting to use treatment units 1 and 4, taking the totality of stream 1 to the output, splitting stream 2 between treatment unit 1 and the output, splitting the third stream between treatment units 1 and 4, and taking all the streams 4 and 5 to unit 1. With the outputs of the treatment unit 1, it all goes straight to the mixer 6 while the output of unit 4 is split between recirculation and back to unit 1.

This returns a total cost of $348,337.04, as seen above, which is the same case for the models provided by the reference when ran in GAMS.

References

Ruiz, J. (2009, May 15). Water Treatment Network Design. Retrieved from MINLP Cyber-Infrastructure: https://www.minlp.org/library/problem/index.php?i=24&lib=MINLP

In [ ]: